EVIDENCIAS DE PRUEBAS

Inventario: revisión de stock con n8n y Supabase

Caso normal, rutas de decisión, fallo técnico controlado y recuperación

Proyecto	Inventario: revisión de stock y alertas de reposición
Curso	Automatización inteligente de procesos con n8n. EAM, periodo 2026 II
Corte y tema	Segundo corte. Clase 08: subworkflows, manejo de errores y registros
Fecha de las pruebas	24 de octubre de 2026
Integrantes	Valentina Gutierrez, Jorge Matinez, Gabriela Rios, Michael Aguilera
Workflows evaluados	Principal - Revisión de stock, Preparar producto (subworkflow) y Manejador de errores - Inventario

1. Propósito del documento

Este documento reúne las evidencias de las pruebas realizadas a la solución de revisión de stock. Para cada caso se registran los datos de entrada, la forma en que se ejecutó, el resultado esperado, el resultado observado y la conclusión, y cada captura se relaciona con el caso que demuestra.

Todas las pruebas usan datos ficticios cargados en una base de datos creada para este ejercicio. Las capturas no contienen claves ni credenciales.

2. Descripción breve de la solución

La solución recibe por un Webhook el nombre de un producto (GET /revisar-stock?producto=...). Consulta la tabla stock_productos de Supabase con un nodo HTTP Request y envía los datos a un subworkflow que los limpia y valida. Con los datos ya validados calcula la prioridad de reposición y, si el producto lo requiere, registra una alerta en la tabla alertas_reposicion. Si ocurre un fallo técnico, un workflow independiente lo guarda en la tabla incidentes.

2.1 Componentes

Componente	Responsabilidad
Principal - Revisión de stock	Recibe la petición por Webhook, consulta el stock, decide la ruta, registra la alerta y responde al cliente.
Preparar producto (subworkflow)	Recibe id, producto, stock y stock_minimo. Elimina espacios, convierte el stock a número y valida los datos. Devuelve datos_validos y, si hay error, el motivo.
Manejador de errores - Inventario	Se activa con Error Trigger cuando falla una ejecución automática del principal. Prepara el incidente y lo guarda en la tabla incidentes.
Supabase (API REST)	Lectura de stock_productos. Escritura en alertas_reposicion e incidentes mediante HTTP Request (POST).

2.2 Reglas de decisión

Condición	Estado devuelto	Acción en el sistema
El producto no existe en la tabla	Producto no encontrado	No se calcula ni se escribe nada.
Stock o stock mínimo vacío, no numérico o negativo	Datos de stock no válidos	El subworkflow rechaza el dato. No se calcula ni se escribe.
Stock menor que el mínimo	Stock crítico (prioridad Alta)	Se registra una alerta en alertas_reposicion.
Stock igual al mínimo	Stock bajo (prioridad Mediana)	Se registra una alerta en alertas_reposicion.
Stock mayor que el mínimo	Stock suficiente (prioridad Baja)	No se registra alerta.

> **Respuesta del negocio y error técnico**
> «Producto no encontrado», «Datos de stock no válidos» y «Stock suficiente» son resultados normales del proceso. Un error técnico es, por ejemplo, que Supabase no encuentre la tabla o no responda. Solo los errores técnicos activan el manejador.

3. Entorno y datos de prueba

- n8n en ejecución local (`localhost:5678`), con los tres workflows publicados para las pruebas automáticas.
- Supabase con tres tablas: stock_productos, alertas_reposicion e incidentes.
- URL de prueba (`/webhook-test/`): ejecución manual desde el editor, tras pulsar Listen for test event.
- URL de producción (`/webhook/`): ejecución automática, con el workflow publicado.

Producto (ficticio)	Stock	Mínimo	Resultado esperado
Peluche Stitch	3	10	Alta: stock crítico
Peluche Mickey	10	10	Mediana: stock bajo
Camiseta Roja	50	10	Baja: stock suficiente
Taza Blanca	8	5	Baja (no se usó en estas pruebas)
Producto Prueba Sin Stock	vacío (null)	5	Datos de stock no válidos

4. Resumen de resultados

N.º	Caso	Entrada	Ejecución	Resultado
1	**Caso normal: stock crítico**	producto=Peluche Stitch	Manual (webhook-test)	**Cumple**
2	**Repetir sin duplicar**	La misma petición del caso 1	Manual (webhook-test)	**Cumple**
3	**Stock bajo (límite)**	producto=Peluche Mickey	Manual (webhook-test)	**Cumple**
4	**Stock suficiente**	producto=Camiseta Roja	Manual (webhook-test)	**Cumple**
5	**Producto no encontrado**	producto=zzzz	Manual (webhook-test)	**Cumple**
6	**Entrada no válida**	producto=Sin Stock (stock vacío)	Manual (webhook-test)	**Cumple**
7	**Fallo técnico controlado**	producto=Peluche Stitch con URL de escritura alterada	Automática (producción)	**Cumple**
8	**Recuperación**	producto=Peluche Stitch con la URL corregida	Automática (producción)	**Cumple**

5. Pruebas del flujo normal y de las rutas de decisión

5.1 Caso 1: caso normal con stock crítico

Objetivo	Comprobar el recorrido completo cuando un producto está por debajo de su stock mínimo.
Datos de entrada	`GET /revisar-stock?producto=Peluche Stitch` Datos ficticios: stock 3, stock mínimo 10.
Tipo de ejecución	Manual, con la URL de prueba (webhook-test).
Pasos	1. Vaciar la tabla alertas_reposicion. 2. Pulsar Listen for test event en el nodo Webhook. 3. Abrir la dirección en el navegador. 4. Revisar la respuesta y la tabla en Supabase.
Resultado esperado	Prioridad Alta, estado «Stock crítico», alerta creada con faltante 7 y alerta_registrada en true.
Resultado observado	La respuesta devuelve alerta_id 1-2026-10-24, faltante 7, prioridad Alta, estado «Stock crítico», acción «Reponer inmediatamente», estado_atencion «pendiente» y alerta_registrada true. La tabla muestra una fila de Peluche Stitch.
Resultado	**Cumple**



Figura 1. *Llamada al Webhook desde el navegador con la URL de prueba.*

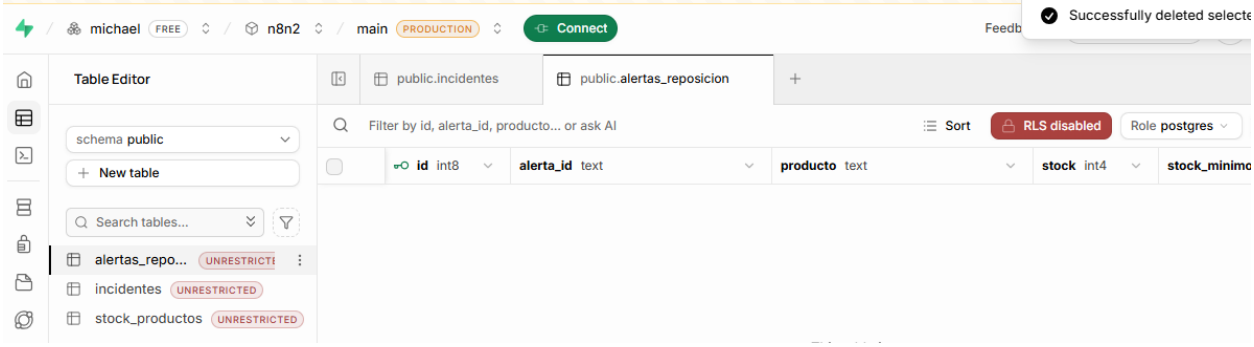


Figura 2. *Tabla alertas_reposicion antes de la prueba, sin filas.*

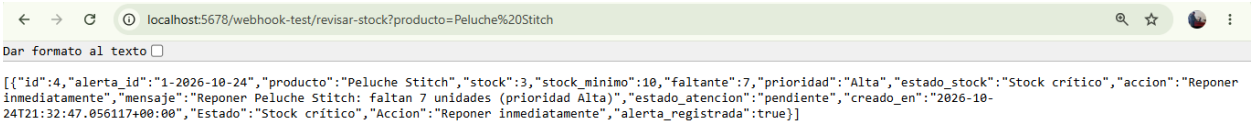


Figura 3. *Respuesta del flujo a la petición del caso 1.*

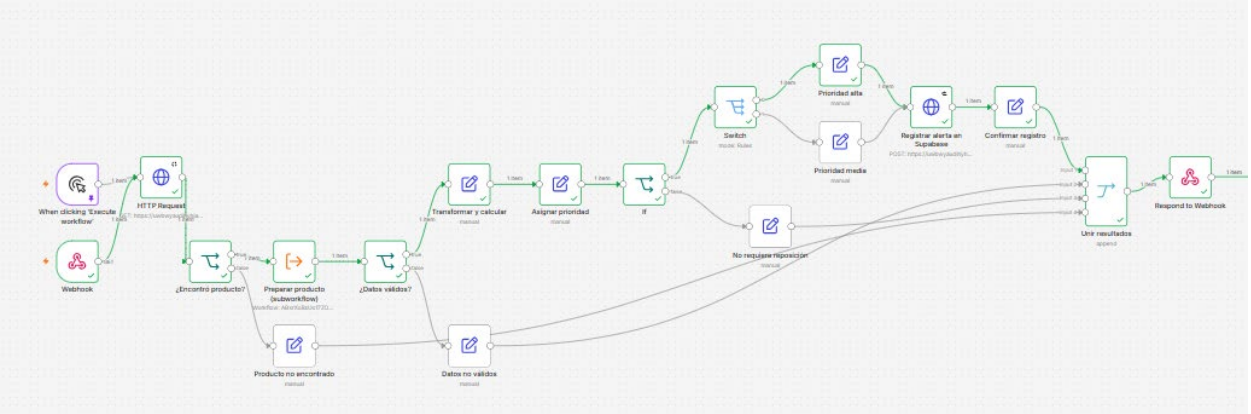


Figura 4. *Recorrido de la ejecución en n8n: la lectura, el subworkflow, el cálculo, la prioridad Alta, el registro de la alerta y la respuesta quedan en verde.*

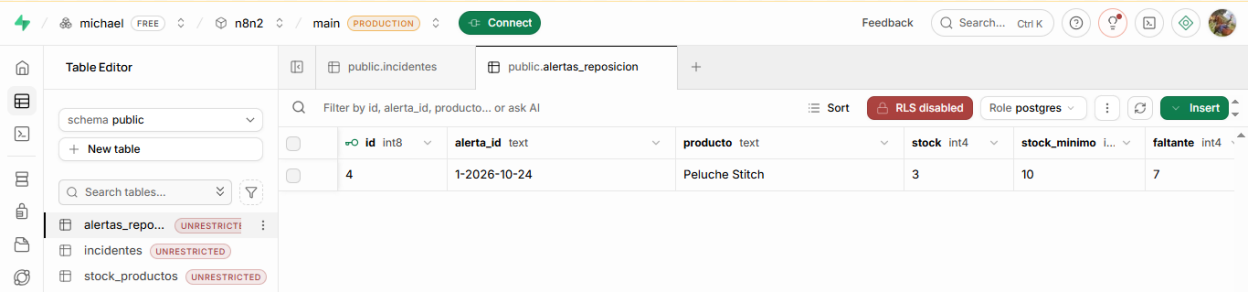


Figura 5. *Tabla alertas_reposicion después de la prueba, con la alerta de Peluche Stitch.*

Cómo leer la respuesta

Campo	Significado
alerta_id: 1-2026-10-24	Clave de la alerta: identificador del producto más la fecha. Evita duplicados el mismo día.
faltante: 7	Unidades que faltan para llegar al mínimo (10 menos 3).
prioridad / estado_stock	Alta y «Stock crítico», porque el stock es menor que el mínimo.
estado_atencion: pendiente	Estado inicial de la alerta, a la espera de que alguien la atienda.
alerta_registrada: true	Se marca en true solo si Supabase devolvió el alerta_id, es decir, si la creación quedó confirmada.

> **Sobre el id 4 de la tabla**
> El contador id de la tabla continúa aunque se borren filas de pruebas anteriores. Por eso la primera alerta de esta prueba aparece con id 4. No indica que falten registros.

5.2 Caso 2: repetir la petición sin duplicar

Objetivo	Demostrar que repetir la misma llamada no crea una segunda alerta.
Datos de entrada	`GET /revisar-stock?producto=Peluche Stitch` La misma petición del caso 1, ejecutada otra vez el mismo día.
Tipo de ejecución	Manual, con la URL de prueba (webhook-test).
Pasos	1. Pulsar de nuevo Listen for test event. 2. Abrir la misma dirección. 3. Recargar la tabla alertas_reposicion.
Resultado esperado	No se crea una segunda fila. La alerta existente se actualiza gracias a on_conflict=alerta_id.
Resultado observado	La tabla conserva una sola fila de Peluche Stitch (id 4, alerta_id 1-2026-10-24).
Resultado	**Cumple**

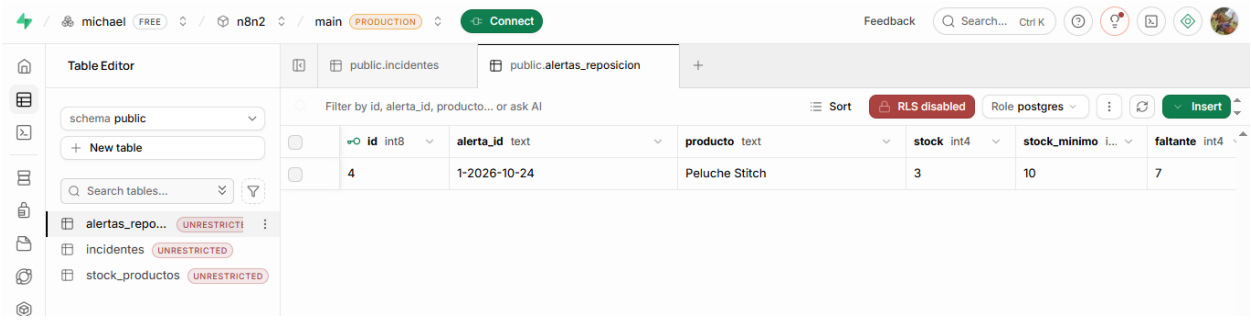


Figura 6. *Tabla alertas_reposicion después de repetir la petición: sigue habiendo una sola fila de Peluche Stitch.*

> **Por qué no se duplica**
> La petición usa POST con el parámetro on_conflict=alerta_id y el encabezado Prefer: resolution=merge-duplicates. Si el alerta_id ya existe, Supabase actualiza la fila. La petición no envía estado_atencion, así que una alerta ya marcada como «resuelta» no vuelve a «pendiente».

5.3 Caso 3: stock bajo en el límite (prioridad Mediana)

Objetivo	Comprobar la ruta Mediana, que se aplica cuando el stock es igual al mínimo.
Datos de entrada	`GET /revisar-stock?producto=Peluche Mickey` Datos ficticios: stock 10, stock mínimo 10.
Tipo de ejecución	Manual, con la URL de prueba (webhook-test).
Pasos	1. Pulsar Listen for test event. 2. Abrir la dirección. 3. Revisar la respuesta y la tabla.
Resultado esperado	Prioridad Mediana, estado «Stock bajo», acción «Reponer pronto», alerta creada. El faltante es 0 porque el stock ya iguala al mínimo.
Resultado observado	La respuesta devuelve alerta_id 2-2026-10-24, faltante 0, prioridad Mediana, estado «Stock bajo», acción «Reponer pronto» y alerta_registrada true. La tabla ahora tiene dos filas.
Resultado	**Cumple**

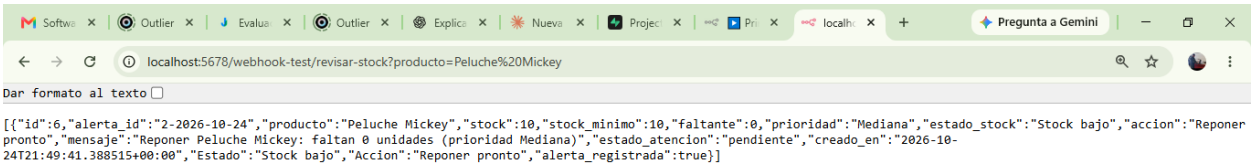


Figura 7. *Respuesta del flujo para Peluche Mickey.*

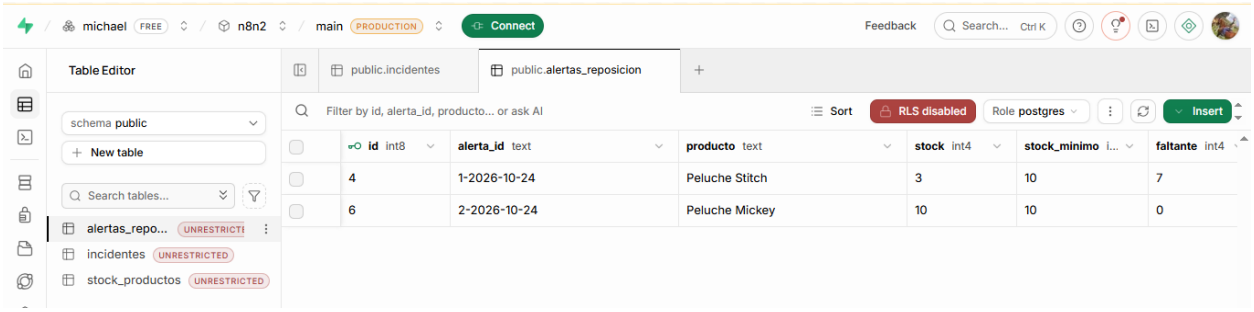


Figura 8. *Tabla alertas_reposicion con las alertas de Peluche Stitch y Peluche Mickey.*

5.4 Caso 4: stock suficiente

Objetivo	Comprobar que un producto con stock por encima del mínimo no genera alerta.
Datos de entrada	`GET /revisar-stock?producto=Camiseta Roja` Datos ficticios: stock 50, stock mínimo 10.
Tipo de ejecución	Manual, con la URL de prueba (webhook-test).
Pasos	1. Pulsar Listen for test event. 2. Abrir la dirección. 3. Comprobar que la tabla no cambia.
Resultado esperado	Prioridad Baja, estado «Stock suficiente», acción «No requiere reposición» y ninguna fila nueva.
Resultado observado	La respuesta indica datos_validos true, prioridad Baja, estado «Stock suficiente» y acción «No requiere reposición». No contiene alerta_id, y la tabla sigue con dos filas (Figura 12).
Resultado	**Cumple**

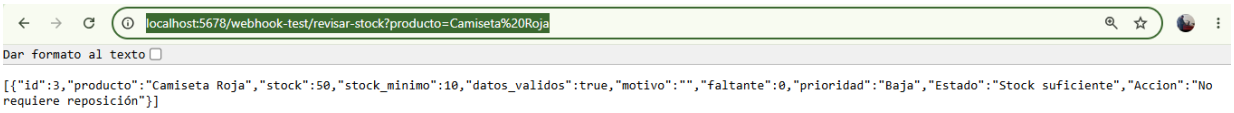


Figura 9. *Respuesta del flujo para Camiseta Roja.*

5.5 Caso 5: producto no encontrado

Objetivo	Comprobar la respuesta cuando el nombre buscado no existe.
Datos de entrada	`GET /revisar-stock?producto=zzzz` Texto que no corresponde a ningún producto.
Tipo de ejecución	Manual, con la URL de prueba (webhook-test).
Pasos	1. Pulsar Listen for test event. 2. Abrir la dirección. 3. Revisar la respuesta.
Resultado esperado	Estado «Producto no encontrado» y acción «Revise el nombre del producto», sin cálculos ni escrituras.
Resultado observado	La respuesta contiene únicamente el estado «Producto no encontrado» y la acción «Revise el nombre del producto».
Resultado	**Cumple**



Figura 10. *Respuesta del flujo para un producto inexistente.*

Es una respuesta del negocio y no un fallo técnico: el flujo funcionó correctamente y la consulta simplemente no encontró coincidencias.

5.6 Caso 6: entrada no válida (stock vacío)

Objetivo	Comprobar que el subworkflow rechaza datos inválidos antes de calcular o escribir.
Datos de entrada	`GET /revisar-stock?producto=Sin Stock` Coincide con «Producto Prueba Sin Stock», cuyo stock está vacío (null) y cuyo mínimo es 5.
Tipo de ejecución	Manual, con la URL de prueba (webhook-test).
Pasos	1. Pulsar Listen for test event. 2. Abrir la dirección. 3. Revisar la respuesta y comprobar que la tabla no cambia.
Resultado esperado	datos_validos en false, motivo «stock no válido», estado «Datos de stock no válidos» y ninguna escritura.
Resultado observado	La respuesta devuelve stock null, datos_validos false, motivo «stock no válido», estado «Datos de stock no válidos» y acción «Corregir el registro en inventario». La tabla permanece con dos filas.
Resultado	**Cumple**

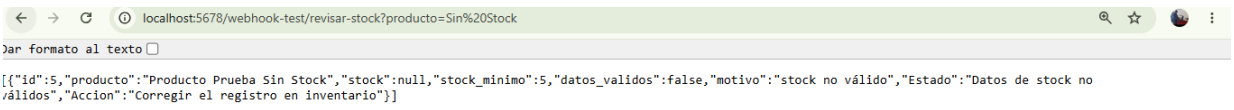


Figura 11. *Respuesta del flujo para un producto con stock vacío.*

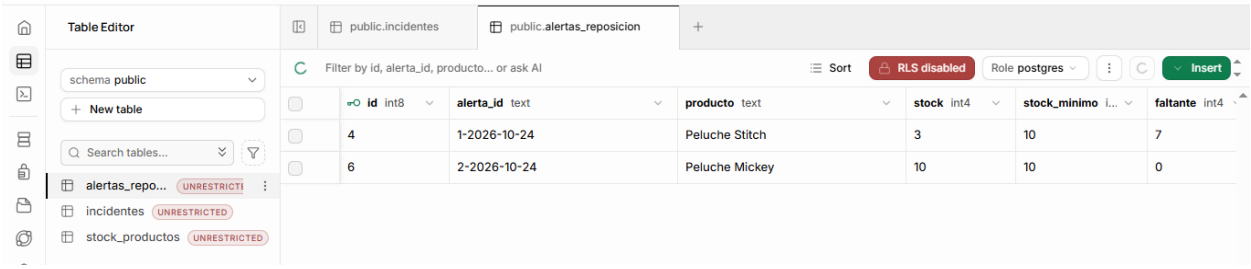


Figura 12. *Tabla alertas_reposicion después de los casos 4, 5 y 6: sigue con dos filas, es decir, ninguno de ellos escribió datos.*

6. Prueba de fallo técnico controlado

6.1 Preparación: publicación y manejador asociado

El Error Trigger solo se activa con ejecuciones automáticas. Por eso, antes de provocar el fallo, fue necesario publicar los tres workflows y asociar el manejador en los ajustes del principal. El orden fue: primero el subworkflow, luego el manejador y al final el principal, porque n8n no permite publicar un workflow que llama a otro sin publicar.

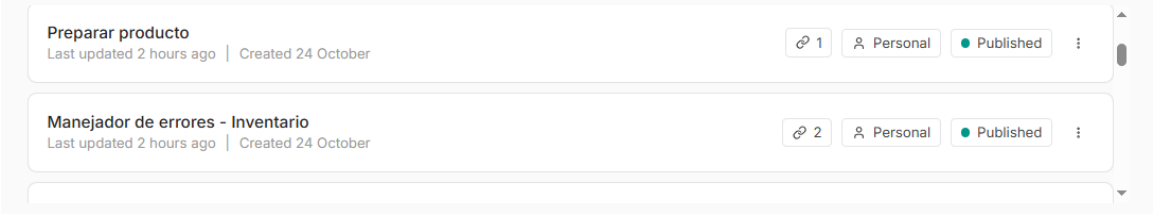


Figura 13. *Subworkflow «Preparar producto» y manejador publicados.*

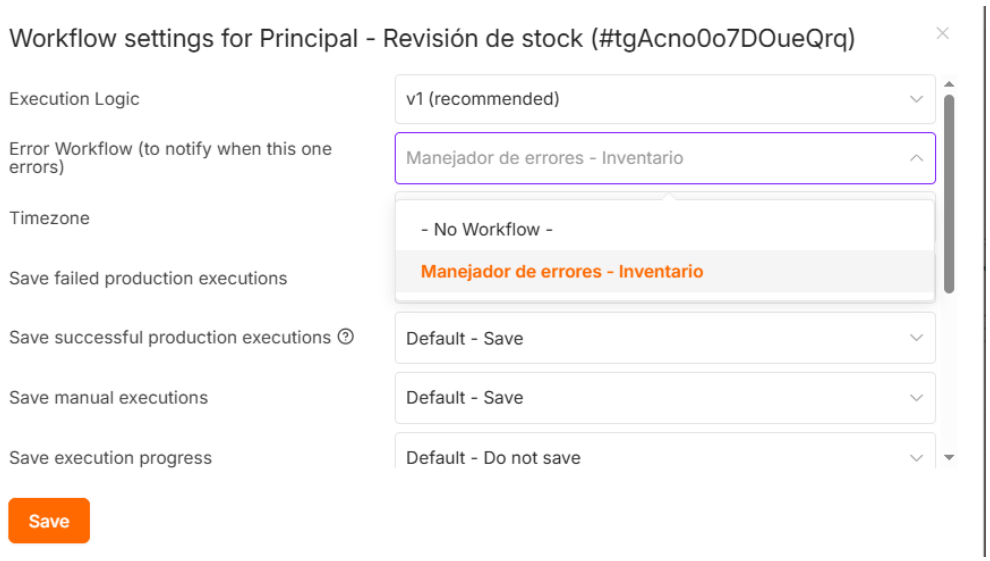


Figura 14. *Ajustes del principal: el manejador ya puede seleccionarse en «Error workflow».*

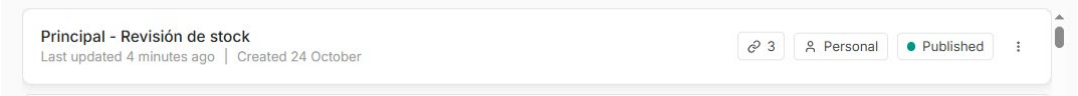


Figura 15. *Workflow principal publicado.*

> **Dificultad encontrada**
> Mientras el manejador no estaba publicado, n8n lo mostraba en gris y no permitía seleccionarlo. El mensaje del triángulo de advertencia indicaba que debía publicarse. Después, el principal tampoco se dejaba publicar hasta publicar el subworkflow que llama.

6.2 Ejecución de la prueba

Objetivo	Comprobar que un fallo técnico no se informa como éxito, que el manejador se activa solo y que el incidente queda guardado en una tabla.
Cómo se provocó	En el nodo «Registrar alerta en Supabase» se cambió la URL de `alertas_reposicion` a `alertas_reposicion_x` (una tabla que no existe) y se publicó de nuevo el principal.
Datos de entrada	`GET /webhook/revisar-stock?producto=Peluche Stitch` URL de producción.
Tipo de ejecución	Automática: workflow publicado y llamada a la URL de producción.
Pasos	1. Comprobar que la tabla incidentes estaba vacía. 2. Cambiar la URL del nodo y publicar. 3. Abrir la dirección de producción. 4. Revisar Executions del principal, Executions del manejador y la tabla incidentes.
Resultado esperado	El nodo falla tras 3 intentos (espera de 2 s entre intentos), no se llega a «Confirmar registro», el manejador se ejecuta y aparece una fila en incidentes.
Resultado observado	• La página del navegador no recibe la respuesta normal del flujo. • El principal registra ejecuciones en estado Error de cerca de 6,5 s, cifra coherente con 3 intentos y esperas de 2 s. El nodo «Registrar alerta en Supabase» queda en rojo con el mensaje «The resource you are requesting could not be found». • El manejador registra ejecuciones exitosas pocos segundos después de cada fallo automático. Por ejemplo, el error de las 18:18:24 del principal tiene su ejecución del manejador a las 18:18:31. • La tabla incidentes contiene filas con el proyecto, el workflow «Principal - Revisión de stock» y el nodo «Registrar alerta en Supabase».
Resultado	**Cumple**

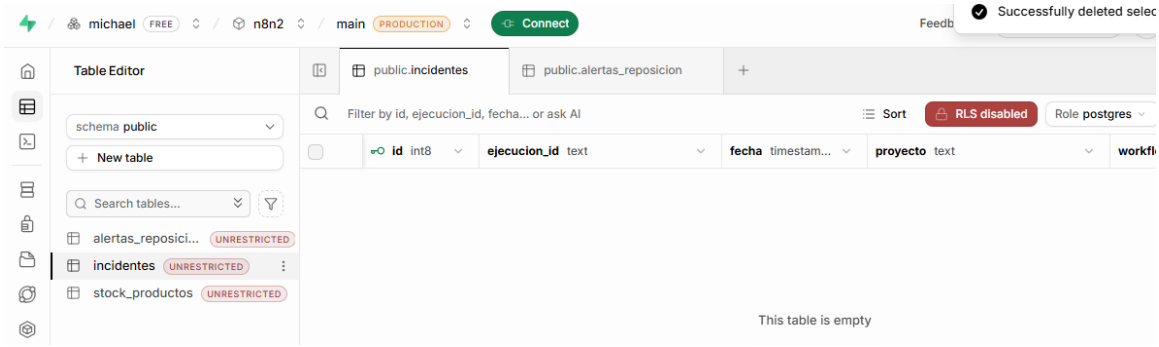


Figura 16. *Tabla incidentes antes de la prueba, vacía.*

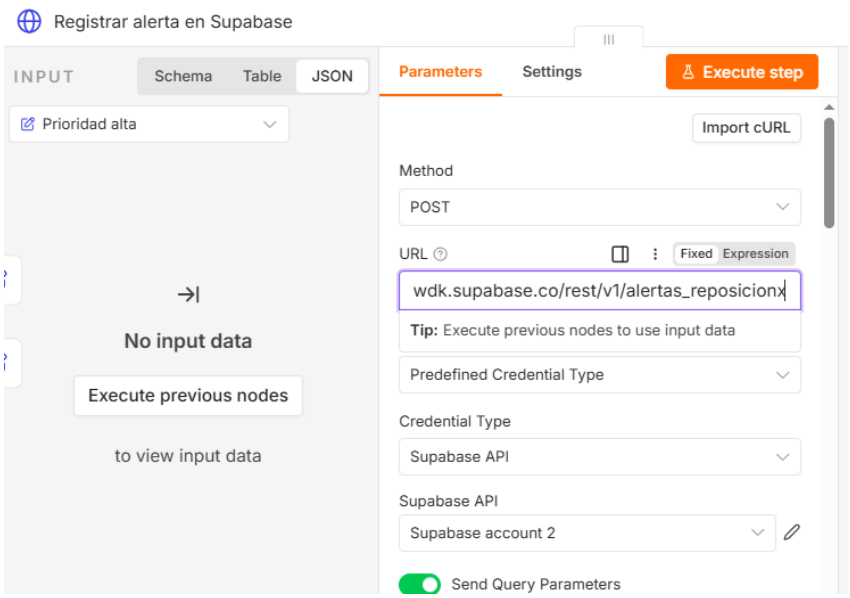


Figura 17. *URL del nodo alterada a una tabla inexistente (alertas_reposicion_x).*



Figura 18. *Llamada a la URL de producción desde el navegador.*

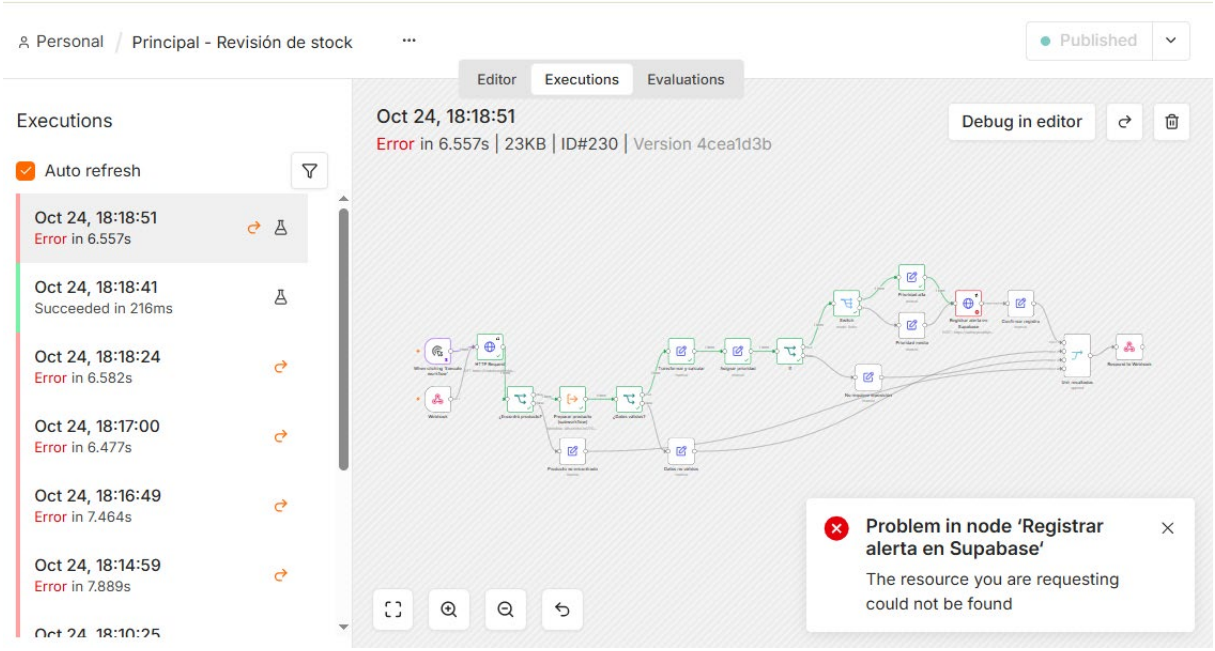


Figura 19. *Executions del principal: ejecuciones en estado Error y mensaje del nodo que falló.*

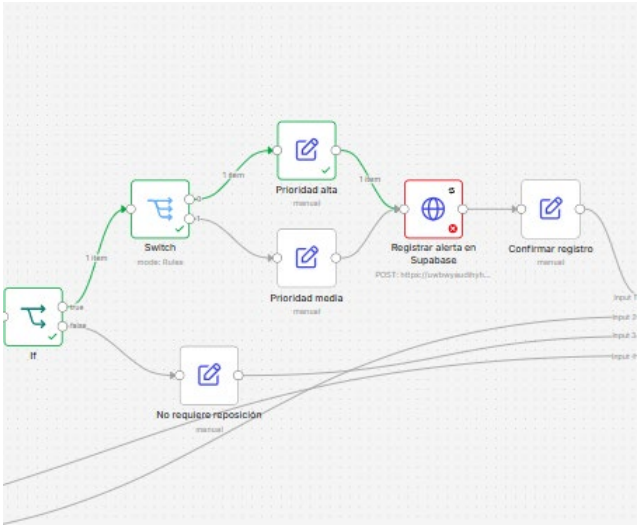


Figura 20. *Nodo «Registrar alerta en Supabase» en rojo; el flujo no llega a «Confirmar registro».*

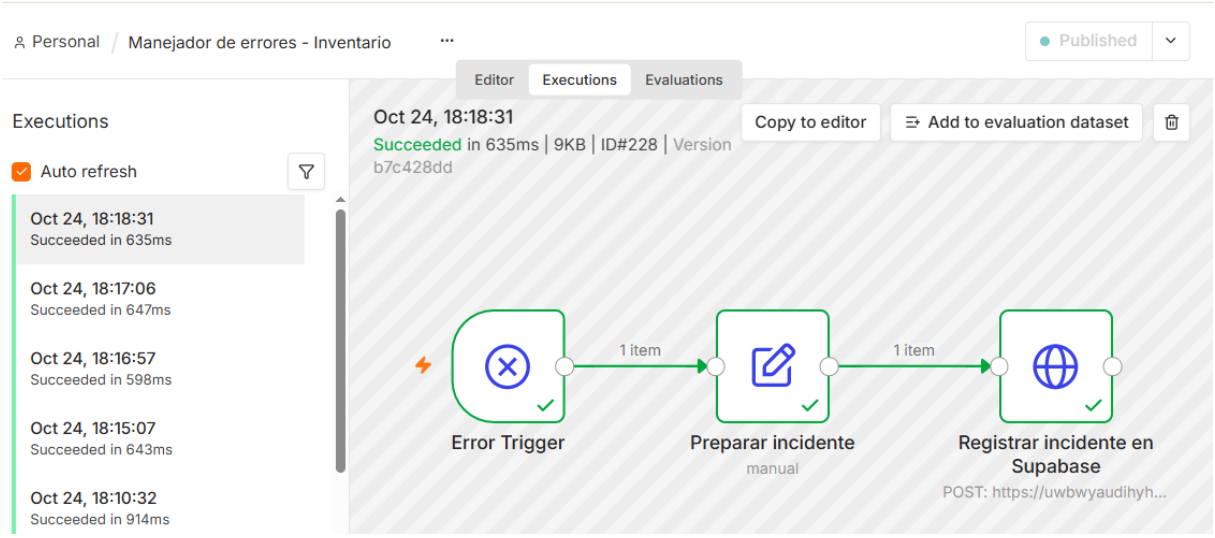


Figura 21. *Executions del manejador: se ejecuta de forma automática después de cada fallo del principal.*

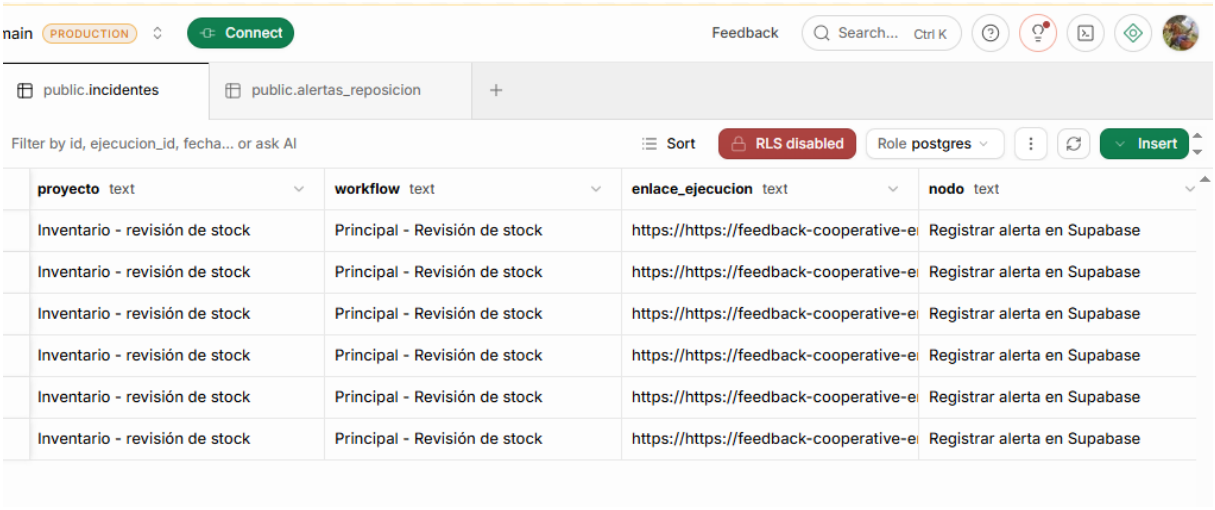


Figura 22. *Tabla incidentes con las filas registradas por el manejador.*

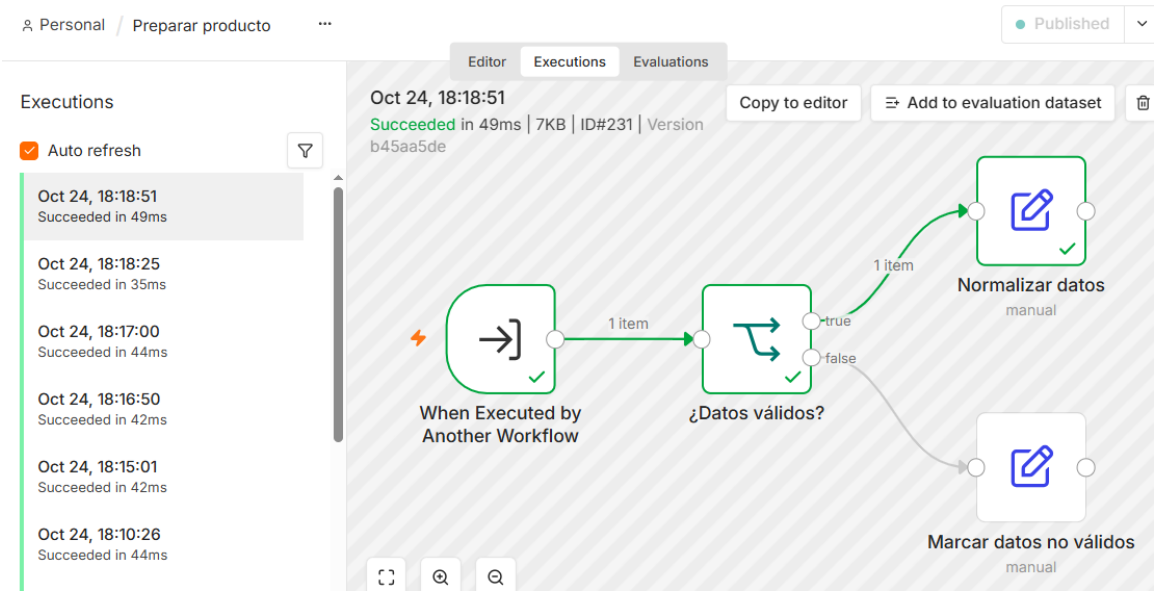


Figura 23. *Executions del subworkflow «Preparar producto»: se ejecutó correctamente antes de que fallara el registro.*

6.3 Observaciones de la prueba

- **Los fallos son técnicos, no del negocio:** la tabla de destino no existe, por lo que el flujo se detiene y no confirma una alerta que no se creó.
- **El canvas del editor no se dibuja en ejecuciones de producción.** Solo las ejecuciones manuales o de prueba se pintan en el editor. Las automáticas se consultan en la pestaña Executions.
- **Una ejecución manual no activa el manejador.** El fallo lanzado desde el editor a las 18:18:51 no aparece en la lista del manejador, lo que concuerda con lo visto en clase.
- **El manejador registra, pero no repara.** No reanuda la ejecución interrumpida ni responde al cliente del webhook.
- **Reintentos configurados:** 3 intentos con 2 segundos de espera en el nodo que escribe la alerta. Es útil ante caídas temporales; no corrige una tabla inexistente ni una credencial inválida.

7. Recuperación

Objetivo	Corregir la causa del fallo, repetir la operación y comprobar que no se duplican registros.
Datos de entrada	`GET /webhook/revisar-stock?producto=Peluche Stitch` La misma petición que falló.
Tipo de ejecución	Automática: workflow publicado y URL de producción.
Pasos	1. Devolver la URL del nodo a alertas_reposicion. 2. Publicar de nuevo el principal. 3. Repetir la misma llamada. 4. Verificar la respuesta, las ejecuciones y la tabla.
Resultado esperado	Respuesta correcta, ejecución exitosa y la misma alerta (mismo alerta_id), sin una segunda fila.
Resultado observado	La respuesta devuelve id 4 y alerta_id 1-2026-10-24, con la misma fecha de creación (21:32:47) de la alerta original, es decir, no se creó una alerta nueva. Las ejecuciones de las 19:19 aparecen como exitosas (3,2 s y 2,5 s).
Resultado	**Cumple**

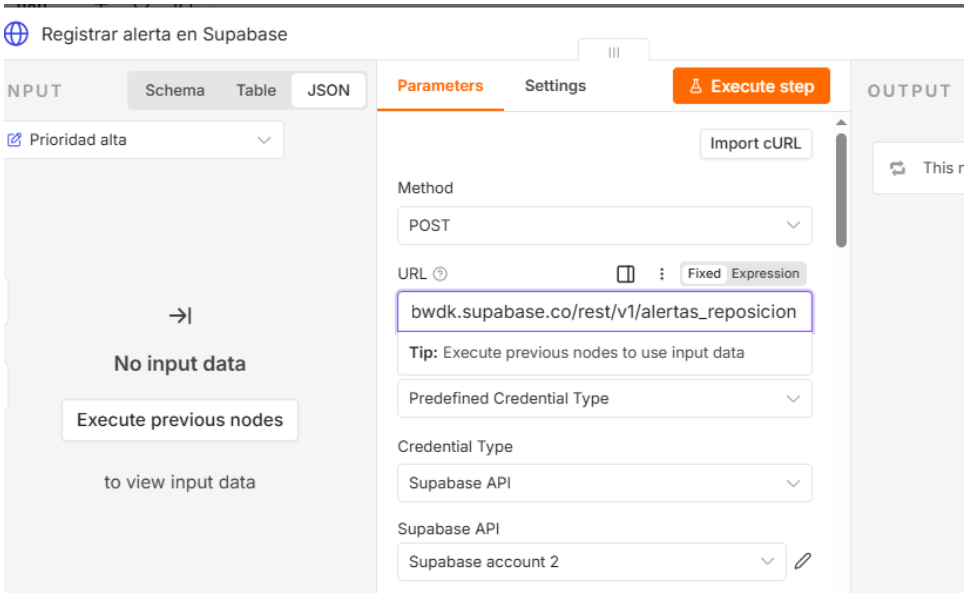


Figura 24. *URL del nodo restaurada a alertas_reposicion.*



Figura 25. *Respuesta correcta tras la corrección: mismo id y misma fecha de creación que la alerta original.*

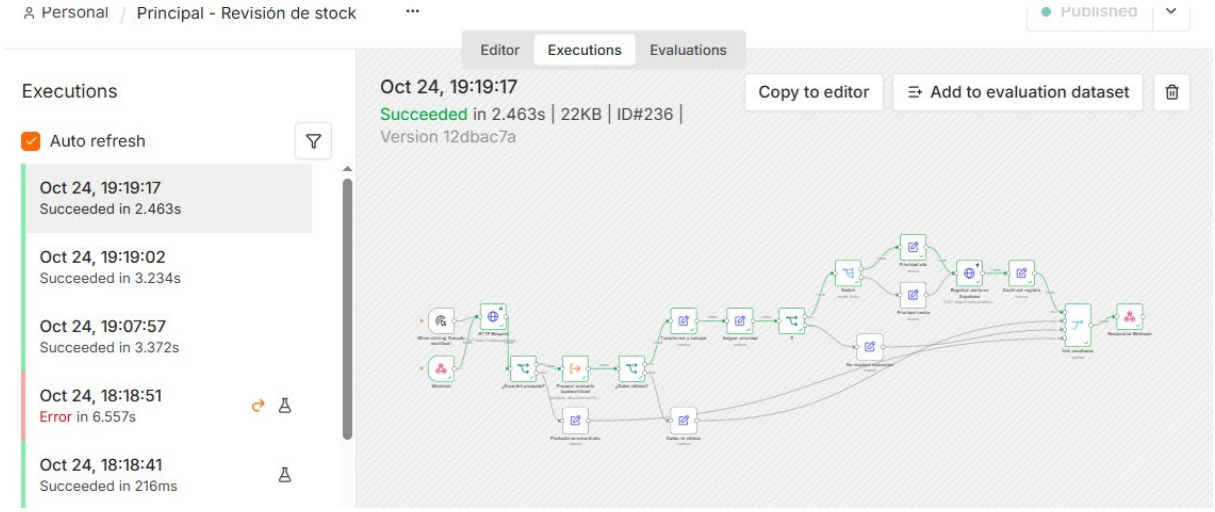


Figura 26. *Executions del principal: las ejecuciones posteriores a la corrección terminan en estado exitoso.*

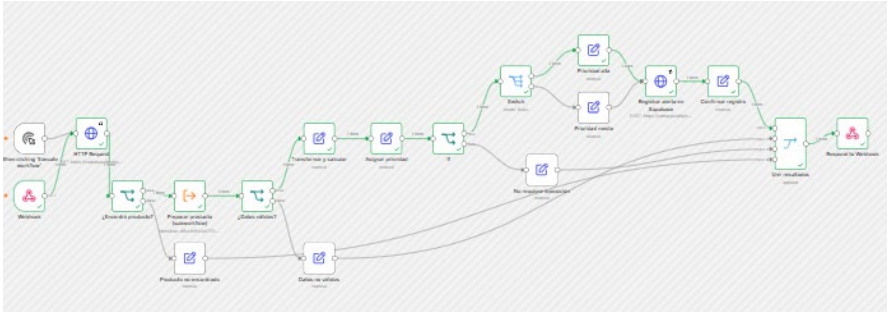


Figura 27. *Recorrido completo de la ejecución de recuperación.*

Criterio de recuperación

Antes de repetir una operación que crea registros se verifica si ya existe. Aquí la clave alerta_id (producto más fecha) y la inserción con fusión de duplicados garantizan que repetir la llamada actualiza la alerta en lugar de crear otra. El incidente solo debe pasar a «resuelto» después de comprobar que la alerta quedó creada.

8. Dificultades encontradas y soluciones

Problema	Causa	Solución
Error 400: «Could not find the '' column of 'incidentes'»	El cuerpo de la petición llegaba con un campo sin nombre.	Se envió el cuerpo como JSON explícito (Using JSON) en el nodo HTTP Request.
Error 400: «failed to parse filter»	Los datos se habían escrito en Query Parameters, que van en la URL, y no en el Body.	Query Parameters conserva solo on_conflict. Los datos van en Send Body, con JSON.
Error 42501: «new row violates row-level security policy»	La seguridad por filas de Supabase bloqueaba la escritura.	Se deshabilitó RLS en las tres tablas del entorno de pruebas. En un entorno real se recomienda mantenerla con políticas.
El manejador aparecía en gris y no se podía seleccionar	No estaba publicado.	Se publicó el manejador y luego se seleccionó en «Error workflow» del principal.
No se podía publicar el principal	Llamaba a un subworkflow que no estaba publicado.	Orden de publicación: subworkflow, manejador y principal.
El canvas no se pintaba en las ejecuciones de producción	El editor solo dibuja ejecuciones manuales o de prueba.	Consultar la pestaña Executions de cada workflow.
El dato fijado del Manual Trigger no filtraba por producto	El HTTP Request lee query.producto, pero el dato fijado no tenía esa estructura.	Se corrigió a {query: {producto: «Peluche Stitch»}}.

9. Estado final de la solución

Categoría	Detalle
Implementado	• Webhook GET que recibe el producto. • Lectura de stock_productos con HTTP Request y escritura de alertas e incidentes con HTTP Request (POST). • Subworkflow «Preparar producto» con entradas y salidas definidas. • Reglas de decisión: no encontrado, datos no válidos, prioridad Alta, Mediana y stock suficiente. • Alerta de reposición sin duplicados (clave alerta_id e inserción con fusión). • Reintentos en el registro de la alerta (3 intentos, 2 s). • Manejador de errores con registro persistente en la tabla incidentes. • Pruebas manuales y automáticas con evidencias.
Simulado	• El fallo técnico se provocó a propósito, cambiando la URL de escritura a una tabla inexistente. • Todos los productos y cantidades son datos ficticios.
Propuesto	• Enviar una notificación (correo o chat) cuando se registre un incidente. Hoy solo se guarda en la tabla. • Responder al cliente del webhook con un mensaje controlado cuando falle el registro. • Agregar al incidente el producto consultado para relacionarlo con la solicitud. • Reactivar RLS con políticas en lugar de dejarla deshabilitada.

•